

Prepare to choose a data store in Azure

When you prepare your landing zone environment for your cloud adoption, you need to determine the data requirements for hosting your workloads. Azure database products and services support various data storage scenarios and capabilities. How you configure your landing zone environment to support your data requirements depends on your workload governance, technical, and business requirements.

Identify data services requirements

As part of your landing zone evaluation and preparation, you need to identify the data stores that your landing zone needs to support. This process involves assessing each of the applications and services that make up your workloads to determine their data storage and access requirements. After you identify and document these requirements, you can create policies for your landing zone to control allowed resource types based on your workload needs.

For each application or service that you deploy to your landing zone environment, use the following information as a starting point to help you determine the appropriate data store services to use.

Functional requirements

Consider the nature of your data and how you plan to use it:

- **Data format:** Structured (tables), semi-structured (JSON, XML, and key-value), or unstructured (images and documents)
- **Purpose:** Online transactional processing (OLTP) for transactional data or online analytical processing (OLAP) for complex, ad-hoc data analysis
- **Search needs:** Indexing capability or full-text search capability
- **Specialized:** Vector stores for highly dimensional data or graph databases for highly interconnected data
- **Data access method:** Direct query language (such as SQL or Gremlin), REST API, SDK, or a combination. Some platforms restrict access to a proprietary API layer, which affects query flexibility and integration options.
- **Data relationships:** Joins, graph traversal, or hierarchical structures
- **Consistency model:** Strong, eventual, or configurable consistency
- **Schema flexibility:** Schema-on-write (rigid) versus schema-on-read (flexible)
- **Concurrency needs:** Optimistic versus pessimistic locking, and high-write scenarios
- **Data life cycle:** Short-lived versus long-term archival, and hot versus cold data
- **Data movement:** Extract, transform, and load (ETL) requirements; extract, load, and transform (ELT) requirements; and integration with pipelines

Nonfunctional requirements

Evaluate performance and scalability expectations:

- **Latency and throughput:** Real-time versus batch processing
- **Scalability:** Vertical versus horizontal scaling, and global distribution
- **Reliability and availability:** Service-level agreement (SLA) requirements and failover strategies
- **Limits:** Storage size, throughput caps, and partitioning constraints

Cost and management considerations

Factor in operational overhead and budget:

- **Managed versus self-hosted:** Software as a service (SaaS), platform as a service (PaaS), and infrastructure as a service (IaaS) trade-offs across control, operational overhead, and flexibility
- **Region availability:** Data residency and compliance needs
- **Cost optimization:** Tiered storage, partitioning, and caching
- **Licensing and portability:** Vendor lock-in and open-source compatibility

Security and governance

Ensure alignment with organizational policies:

- **Encryption:** At-rest and in-transit encryption
- **Authentication and authorization:** Role-based access and identity integration
- **Auditing and monitoring:** Activity logs, alerts, and diagnostics
- **Networking:** Private endpoints, firewall rules, and virtual network integration

DevOps and team readiness

Assess your team's ability to support and evolve the solution:

- **Skill sets:** Familiarity with query languages, SDKs, and tooling
- **Client support:** Language bindings and driver availability
- **Tooling integration:** Continuous integration and continuous delivery (CI/CD) pipelines and observability tools

Key questions

Answer the following questions about your workloads to make decisions based on the Azure database services decision tree:

- **What level of control do you need over the OS and database engine?** Some scenarios require you to have a high degree of control or ownership of the software configuration and host servers for your database workloads. In these scenarios, you can deploy custom IaaS virtual machines (VMs) to fully control the deployment and configuration of data services. You might not need this level of control, but maybe you're not ready to move to a full PaaS solution. In that case, a managed instance can provide higher compatibility with your on-premises database engine while providing the benefits of a managed platform.

- **Will your workloads use a relational database technology?** If so, choose from [Azure SQL Database](#), [Azure Database for MySQL](#), and [Azure Database for PostgreSQL](#), which all provide managed PaaS database capabilities.
- **Will your workloads use SQL Server?** In Azure, your workloads can run on IaaS-based [SQL Server on Azure Virtual Machines](#) or on the PaaS-based [SQL Database hosted service](#). Your choice depends on whether you want to manage your database, apply patches, and take backups, or delegate these operations to Azure. Some scenarios require IaaS-hosted SQL Server because of capability requirements. For more information, see [Choose the right SQL Server option in Azure](#).
- **Does your Azure solution include Power Platform or Dynamics 365 workloads?** [Microsoft Dataverse](#) is the SaaS data platform for Power Platform and Dynamics 365 business applications. Unlike the PaaS and IaaS database services in this article, Dataverse abstracts away all infrastructure and engine management. It provides a relational data store with built-in business rules, granular security (role-based, row-level, and column-level), and a standardized [Common Data Model](#). Client applications primarily access data through Dataverse APIs rather than direct SQL connections to an underlying database engine, which limits query flexibility and throughput compared to Azure-native database services. Evaluate Dataverse for the portions of your solution that use Power Apps, Power Automate, Power Pages, or Dynamics 365. For components of the same solution that require high-throughput processing, complex analytical queries, or direct database control, use an Azure-native database service alongside Dataverse. You can synchronize data between Dataverse and Azure services by using tools such as [Azure Synapse Link for Dataverse](#) and [Microsoft Fabric](#).
- **Will your workloads use key-value database storage?** [Azure Managed Redis](#) is a managed in-memory data store based on the latest Redis Enterprise version. It provides low latency and high throughput. [Azure Cosmos DB](#) also provides key-value storage capabilities.
- **Will your workloads use document or graph data?** [Azure Cosmos DB](#) is a multimodel database service that supports various data types and APIs. It also provides document and graph database capabilities. [Azure DocumentDB](#) is a fully managed, open-source, MongoDB-compatible database service.
- **Will your workloads use column-family data?** [Azure Managed Instance for Apache Cassandra](#) provides a managed Apache Cassandra cluster that can extend your existing datacenters into Azure or serve as a cloud-only cluster and datacenter.
- **Will your workloads require high-capacity data analytics capabilities?** [Microsoft Fabric](#) is an enterprise-ready, end-to-end analytics platform. It unifies data movement, data processing, ingestion, transformation, real-time event routing, and report building.
- **Will your workloads require search engine capabilities?** You can use [Azure AI Search](#) to build AI-enhanced cloud-based search indexes that can integrate into your applications.
- **Will your workloads use time-series data?** [Azure Data Explorer](#) is a managed, high-performance, big data analytics platform that analyzes high volumes of data in near real time.

 Note

For more information about how to assess database options for each of your applications or services, see [Understand data store models](#).

Common database scenarios

The following table lists common use scenario requirements and the recommended database services to handle them.

 Expand table

Your goal	Recommended database service
Build apps that scale with a managed and intelligent SQL database in the cloud.	SQL Database
Modernize SQL Server applications by using a managed, up-to-date SQL instance in the cloud.	Azure SQL Managed Instance
Migrate your SQL workloads to Azure while maintaining complete SQL Server compatibility and OS-level access.	SQL Server on Virtual Machines
Build scalable, managed enterprise-ready apps on open-source PostgreSQL, scale out single-node PostgreSQL with high performance, or migrate PostgreSQL and Oracle workloads to the cloud.	Azure Database for PostgreSQL
Deliver high availability and elastic scaling to open-source mobile and web apps by using a managed community MySQL database service, or migrate MySQL workloads to the cloud.	Azure Database for MySQL
Build applications that have guaranteed low latency and high availability anywhere, at any scale, or migrate Cassandra, Gremlin, and other NoSQL workloads to the cloud.	Azure Cosmos DB
Migrate MongoDB workloads to the cloud, or build hybrid and multicloud applications with high-capacity vertical and horizontal scaling	Azure DocumentDB
Modernize existing Cassandra data clusters and apps, and gain flexibility by using a managed instance service.	Azure Managed Instance for Apache Cassandra
Deliver fast, scalable applications by using an open-source-compatible in-memory data store.	Azure Managed Redis
Build or extend business applications by using Power Platform or Dynamics 365 with built-in data modeling, business logic, and security.	Microsoft Dataverse

Database and data platform feature comparison

The following table lists features available in Azure data services and Microsoft Dataverse.

 Expand table

Feature	SQL Database	SQL Managed Instance	Azure Database for PostgreSQL	Azure Database for MySQL	Azure Managed Instance for Apache Cassandra	Azure Cosmos DB	Azure Managed Redis	Azure Document
Database type	Relational	Relational	Relational	Relational	NoSQL	NoSQL	In-memory	NoSQL

Feature	SQL Database	SQL Managed Instance	Azure Database for PostgreSQL	Azure Database for MySQL	Azure Managed Instance for Apache Cassandra	Azure Cosmos DB	Azure Managed Redis	Azure Document
Data model	Relational	Relational	Relational	Relational	Wide-column	Multimodel: Document, wide-column, key-value, graph	Key-value	Document
Distributed multiprimary writes	No	No	No	No	Yes	Yes	Yes	Yes
Virtual network connectivity support	Virtual network service endpoint	Native virtual network implementation	Virtual network injection	Virtual network injection (Flexible Server only)	Native virtual network implementation	Virtual network service endpoint	Virtual network service endpoint	Virtual network service endpoint

ⓘ Note

[Azure Private Link service](#) simplifies networking design by enabling Azure data services to communicate over private networking. Most Azure database services listed in this table support Azure Private Link through [private endpoints](#). For managed instance database services, these instances are deployed in virtual networks, so you don't need to deploy private endpoints for them. Microsoft Dataverse is hosted on Power Platform rather than as an Azure resource, and its virtual network and private connectivity options are described in [Managed virtual network support for Power Platform](#).

Regional availability

Azure helps you deliver services at the scale needed to reach customers and partners anywhere. When you plan your cloud deployment, determine the Azure region to host your workload resources.

Most Azure regions support most database services. A few regions support only a subset of these products, but they mostly target governmental customers. Before you decide which regions to deploy your database resources to, see [Products available by region](#) to check the latest status of regional availability.

Microsoft Dataverse environments are deployed to a [Power Platform geography](#) (such as United States, Europe, or Australia) rather than to a specific Azure region. Data stays within the chosen geography, but you don't control which Azure region within that geography hosts your environment. Consider this coarser region granularity when you plan colocation with other Azure services or when you have specific latency or [data residency requirements](#).

For more information about Azure global infrastructure, see [Azure geographies](#).

Data residency and compliance requirements

Legal and contractual requirements related to data storage often apply to workloads. These requirements might vary based on the location of your organization, the jurisdiction of the physical assets that host your data stores, and your applicable business sector. Consider the following components of data obligations:

- Data classification
- Data location
- Responsibilities for data protection under the shared responsibility model

For information about these requirements, see [Achieve compliant data residency and security with Azure](#).

Part of your compliance efforts might include controlling where your database resources are physically located. Azure regions are organized into groups called *geographies*. An [Azure geography](#) honors data residency, sovereignty, compliance, and resiliency requirements within geographical and political boundaries. If your workloads are subject to data sovereignty or other compliance requirements, you must deploy your storage resources to regions in a compliant Azure geography.

Establish controls for database services

When you prepare your landing zone environment, you can establish controls that limit the data stores that users can deploy. Controls can help you manage costs and limit security risks. Developers and IT teams can still deploy and configure resources that support your workloads.

After you identify and document your landing zone's requirements, you can use [Azure Policy](#) to control the database resources that you allow users to create. Controls can allow or deny the creation of [database resource types](#).

For example, you might restrict users to creating only SQL Database resources. Use policies to control the options that users can select when they create resources. For example, you can restrict SQL Database SKUs that users can provision by allowing only specific versions of SQL Server to be installed on an IaaS VM. For more information, see [Azure Policy built-in policy definitions](#).

You should apply policies to resources, resource groups, subscriptions, and management groups throughout your cloud estate.

Microsoft Dataverse environments are governed separately through the [Power Platform admin center](#), not through Azure Policy. Use [Managed Environments](#) and [data loss prevention policies](#) to control environment creation, connector usage, and data movement. If your solution spans both Azure-native databases and Dataverse, plan for governance across both control planes.

Next steps

- [Understand data models](#)

Use the following articles to choose a specialized data store:

- [Choose a big data storage technology in Azure](#)
- [Choose a search data store in Azure](#)
- [Choose an Azure service for vector search](#)

Last updated on 03/04/2026